Final Exam (Part 1)

Program Design and Data Structures (1DL201)

Teachers: Johannes Borgström and Eva Darulova

2023-01-10 / 08:00 - 12:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **16 questions**. Each question can award 1 to 4 points. You can obtain a maximum of **38 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- Johannes and Eva will come to the exam hall around 9:00 to answer questions.

Question 1: Expression evaluation (2 points)

What is the result of evaluating the following expression?

```
const x = 5;
x < 10 ? x * x + 1 : x - 3;
```

Answer: 26

Show the steps of evaluation:

```
// 1. substitute value 5 for x in the conditional expression:
5 < 10 ? 5 * 5 + 1 : 5 - 3;
// 2. evaluate predicate
5 < 10 -> true
// 3. replace conditional with consequent expression and evaluate:
5 * 5 + 1 -> 26
```

Question 2: Recursion (2 points)

Recall the pre-defined function `display(v)` that displays the value `v` in the console followed by a newline character (and otherwise does nothing else). What is the sequence of values printed by the `display` function when the following program is evaluated?

```
function f(n) {
     display(n);
     return (n <= 1) ? 1
            : (n === 2) ? 2
            : n * f(n - 1);
}
f(5);
```

Answer:

```
5
4
3
2
```

Does the function `f(n)` give rise to an iterative or a recursive process? **Justify your answer** (of this question only; you do not need to explain the values printed by `display`).

Answer and Justification:

It is a recursive process; multiplication operations accumulate as deferred operations.

Question 3: Scope (1 point)

Consider the following code:

```
1    const x = 42;
2    function f(g, x) {
3        function h(x) {
4            const y = g(x);
5            return g(y) > 0 ? y : g(x);
6        }
7        return h(x);
8    }
```

To which declaration does the (underlined) name `x` on line 5 refer to? You can use the line numbers to identify the wanted declaration.

Answer: The declaration on line 3.

Question 4: Higher-order programming (2 points)

Consider the following function declarations:

```
function f(x, y, z) {
    return x + 2 * y + 3 * z;
}
function g() {
    return z => (y => (x => x + 2 * y + 3 * z));
}
```

Write an expression using the function `g` that produces the same result as `f(7, 8, 9)`. You **must not** use the function `f` in your solution.

Answer: `g()(9)(8)(7);`

Question 5: Lists (1 point)

What is the result of evaluating the following Source program in list notation?

```
const xs = list(6, 7, 8);
pair(xs, tail(xs));
```

Answer: `list(list(6, 7, 8), 7, 8)`

Question 6: Data abstraction (3 points)

In the lectures, we have seen that Source can represent only a finite number of integers with full precision. In this question, we are designing a "big integer" data structure that provides full precision for arbitrarily large integers. The idea is to represent a number using a sign (the string `"+"` or the string `"-"`) and a list of decimal digits, each represented as an ordinary Source number ranging from 0 to 9.

Define the functions `make_big_int`, `get_sign` and `get_digits` that allow to construct and access such big integers. You may use suitable data structures seen in the course.

Here is an example of how the functions should work:

```
const bi = make_big_int("+", list(2,3,4));  // represents 234
get_sign(bi);                               // returns "+"
get_digits(bi);                             // returns list(2,3,4)
```

Answer:

```
function make_big_int(sign, digits) {
    return pair(sign, digits);
}

function get_sign(big_int) {
    return head(big_int);
}

function get_digits(big_int) {
    return tail(big_int);
}
```

Question 7: List processing (4 points)

Complete the definition of the `sublist` function below. You must use the pre-defined template and cannot use a different way to implement the function.

```
function get_sublist(start, end, lst) {
  function helper(pos, ys) {
      if (pos < start) {
          return helper(pos + 1, tail(ys));
      } else if (pos <= end) {
          return pair(head(ys), helper(pos + 1, tail(ys)));
      } else {
          return null;
      }
  }
  return helper(0, lst);
}
```

Question 8: Orders of growth (3 points)

Let `n` denote the length of the input list `lst` in the following function. What is the order of growth in time in `n` **using the Θ (Big Theta) notation** of the `find_ranks` function?

```
function find_ranks(lst) {
    return map(y => length(filter(x => x <= y, lst)),
               lst);
}
```

Answer: $\Theta(n^2)$

Give a justification for your answer:
The `map` function runs through all n elements of `lst` once, and for each such element, the `filter` function runs through all n of the elements of `lst` once as well, and the `length` function runs through at most n elements. In total, the order of growth is $n * (n + n)$, i.e. $\Theta(n^2)$.

Is the function `find_ranks` a good candidate for memoization? **Explain your answer.** It depends. If the input list `lst` contains many repeated elements then memoizing the result of `length(filter(x => x <= y, lst))` can save running time.

Question 9: Recurrences from program (4 points)

Consider the following function

```
function foo(xs) {
    return is_null(xs) ? xs : pair(head(xs), append(foo(tail(xs)), list(head(xs)));
}
```

Give a recurrence that describes the running time of `foo`, as a function of the length of the input.
Use the convention of setting the highest coefficient of polynomials to 1, and lower ones to 0.

Hint: What is the length of the output of `foo` for an input of length n?
And how does that influence the runtime of the body of `foo`?

$T(n) = T(n-1) + n$

Give a justification for your answer:

The only recursive call in the function has the tail of the input list as argument, so the size of the argument shrinks by one in each iteration.
Most of the operations in the body of the function take constant time. The exception is append, which takes time proportional to the length of its first argument.
The first argument is `foo(tail(xs))`. The list returned by `foo` has double the length of the argument, so the call to `append` takes time proportional to n. By convention we write $T(n) = T(n-1) + n$ as the recursion for the running time.

Question 10: Trees (2 points)

Complete the function `negate_bst` that takes in a binary search tree (BST) of numbers and returns a new BST of numbers that has all the numbers from the input BST negated. Ensure that your function actually returns a search tree, i.e., that the smallest element is in the leftmost node.

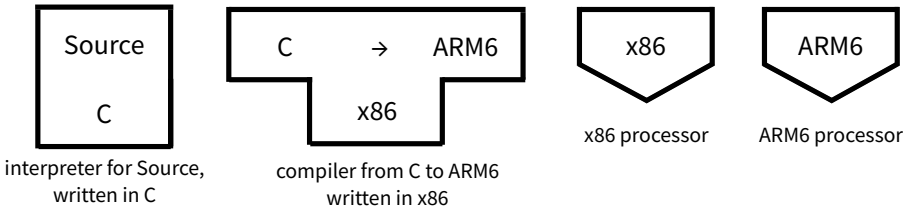
Recall the binary tree functions, augmented with the convenience function `leaf` that creates a new node with the given entry and empty trees as children.

```
is_empty_tree(tree), entry(tree), left_branch(tree),right_branch(tree),
make_empty_tree(), make_tree(entry, lbranch, rbranch), make_leaf(entry)
```

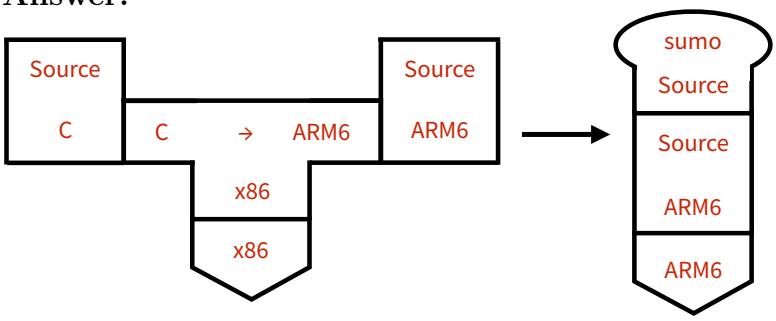
```
function negate_tree(tree) {
  return is_empty_tree(tree)
         ? tree
         : make_tree(-entry(tree),
                     negate_tree(right_branch(tree),
                     negate_tree(left_branch(tree))
}
```

Question 11: Programming language processing (2 points)

We want to write a program `sumo` in Source and run it on a robot that has as ARM6 processor. To do so, we want to use an *interpreter* from Source to ARM6 on the robot itself.
We do have an interpreter for Source, but it is written in the language C, instead of the ARM6 machine code. Fortunately, we have a compiler that can translate programs written in C to programs written in ARM6 machine code. The compiler is written in x86-64, and we have an x86-64 processor. That is, we have the following components:



Complete the following T-diagram of the language processing steps that will allow us to run our Source program `sumo` on the ARM6 processor.
Answer:



Question 12: Mutable data (2 points)

What is the result (in list notation) of evaluating the following program?

```
const A = list(list(2), list(4));
const B = list(list(1), list(3));
set_head(A, head(B));
set_head(head(B), 5);
A;
```

Answer: `list(list(5), list(4))`

Give a justification for your answer:

```
// after the statement
set_head(A, head(B));
// the head of A points to the same head list as B's (list(1))

// the statement
set_head(head(B), 5);
// modifies the list(1) to be list(5), changing both A and B
// hence A now points to list(list(5), list(4))
```

Question 13: Environment model (2 points)

Consider the following Source program:

```
function square(x) {
    return x * x;
}

function compose(f, g) {
    return y => f(g(y));
}
const forth_power = compose(square, square);
forth_power(3);
```

How many function objects get created during the evaluation of this program?

Answer: 3

Give a justification for your answer:
The functions square, compose, and the anonymous function returned by compose.

Question 14: Sorting (2 points)

Recall the **Selection Sort** sorting algorithm for sorting values in increasing order and its **in-place** implementation using arrays. Show the contents of the array as it changes during the run of Selection Sort after each step of the algorithm, i.e. after each (potential) swap.
Answer:

9	2	7	1	3
1	2	7	9	3
1	2	7	9	3
1	2	3	9	7
1	2	3	7	9

Question 15: Arrays (3 points)

The game of tic-tac-toe is played on a 3×3 grid whose 9 slots are initially free. We represent the grid by 2D arrays and use the underscore character `"_"` to represent a free slot. The following grid is completely free:

```
const grid_1 = [ ["_", "_", "_"],
                 ["_", "_", "_"],
                 ["_", "_", "_"] ];
```

Write a function `free_grid` that takes a (possibly not free) 3 x 3 grid (2D array) as input and that resets all slots in the grid to `"_"` so that its content looks like `grid_1` above. Your function should change the input grid and **not create any new arrays**. Your solution may assume that the input array has dimensions 3 x 3.

Answer:

```
function free_grid(grid) {
  for (let r = 0; r < 3; r = r + 1) {
      for (let c = 0; c < 3; c = c + 1) {
          grid[r][c] = "_";
      }
  }
}
```

Question 16: Streams (3 points)

Write a function `skip(intervals, data)` that takes that takes two streams and produces a stream of values from the `data` stream skipping over as many items as given by the corresponding number in the `intervals` stream.

Assume that the data stream is infinite, and that the intervals stream only contains non-negative integers. If the intervals stream is finite, the function should return a stream of the same length as the intervals stream. Ensure that the function only traverses each stream once, and only evaluates the tail function of each pair once.

Examples: `skip(stream(0,1,2,3), integers)` should return `stream(0, 2, 5, 9)`.
`skip(threes, integers)` should return an infinite stream with values 3, 7, 11, 15, ...

```
function skip(intervals, data) {
    function stream_drop(n, str) {
        return n <= 0 ? str : stream_drop(n - 1, stream_tail(str));
    }
    if (is_null(intervals)) {
        return intervals;
    } else {
        const new_data = stream_drop(head(intervals), data);
        return pair(head(new_data), () =>
                    skip(stream_tail(intervals),
                         stream_tail(new_data)));
    }

}
```

This is the final page.